Before Tuning the Harness: Repeat-Controlled Evaluation and Request-Level Accounting for ARC-AGI-3

Guoyi Zhang

College of Sports Medicine and Rehabilitation, Shandong First Medical University & Shandong Academy of Medical Sciences, Tai'an, Shandong, China

zhanguo2-1@outlook.com

Abstract

Evaluating changes to interactive reasoning agents is difficult when each run is costly and outcomes depend on stochastic, long-horizon trajectories. We study this problem on an open ARC-AGI-3 agent built on the Duck harness and Qwen3.8-Flash-Next, a stack that ran mid-pack on the public leaderboard. In three frozen ten-game sessions, control arms of one identical configuration differed by 3.33, 3.20 and 1.23 points depending on the session, and four submissions of one code version produced public scores ranging from 24.38 to 29.60. These observations show why isolated score differences can mislead harness decisions, while the available repetitions are too few to define a general detection threshold. Request-level logs from two 25-game calibration runs add a second measurement issue: 15.0% of analyzer requests hit a 120-second client timeout, and the timeout-terminated requests account for 37.5% of cumulative request duration. Extending the timeout changed the error profile without producing a demonstrated score improvement. We describe a repeat-controlled evaluation protocol for this setting, show with a retrospective audit that its decision rules change how many of our own candidate changes would have been kept, and report harness modifications whose local gains were not resolved by leaderboard evidence. The run artifacts and analysis scripts are packaged for release under MIT-0.

1 Introduction

ARC-AGI-3 is an interactive reasoning benchmark: video-game-like environments where an agent must perceive a board, act on it, and infer the rules while playing. No instructions are given, and the competition form adds hard operational constraints. Submissions run as Kaggle notebooks on a single RTX PRO 6000 with no internet access, for at most nine hours, and prize eligibility requires open-sourcing the method. Whatever intelligence the agent has must fit on one GPU.

This paper is about a smaller question than solving ARC-AGI-3. It is about how you decide whether a change to an agent stack helped. We entered the back half of the competition with a working stack of the kind many teams ran: Tufa Labs' Duck harness, a tool-using solver in which the model writes Python to inspect the board and act, carried forward by Daniel Franzen's milestone-2 solution and served with Qwen3.8-Flash-Next under SGLang. Our public score sat near 30. We had a list of candidate improvements, a small GPU budget, and for several days we made the mistake this paper is about: we believed single comparisons, because a run costs hours and a rerun feels wasteful.

Two observations forced a change of method. A drift control that we added almost as an afterthought, a repeat of the baseline configuration run at the end of the same session, came back 3.33 points above its twin.

And the leaderboard itself scored our byte-identical code at 29.28, 24.38, 29.41 and 29.60 across four submissions. Once those facts sit next to each other, most of the tuning advice in the competition threads becomes unreadable, including our own.

The paper has one method and two case studies. The method is a repeat-controlled evaluation protocol for experiments under a constrained compute budget: frozen game sets, same-session control arms bracketing every comparison, decision rules that report insufficient evidence instead of small wins, and a rule that every number must be recomputable from stored artifacts. The case studies apply it to the two questions we could afford: how large the same-configuration variation is on this benchmark (Section 4), and where the compute budget actually goes (Section 5). Section 6 reports what we tested against the protocol, what the leaderboard could resolve, and what remains unknown.

We did not find a large score improvement, and our entry is mid-pack. The dataset behind the strongest inference in this paper is four leaderboard reruns of one build, three sessions of same-configuration controls, and four screens of a frozen ten-game set. Those are small samples, and Section 8 says so again. What we can offer is a protocol that anyone with the same hardware can run in a few hours, a set of negative results that competition write-ups usually omit, and an account of the compute budget that helps explain why some changes did not show reliable gains in our experiments. The code, the harness patch, the run logs and the analysis scripts will be released before the paper deadline.

2 Related work

Reproducibility and statistical reporting. The practice of comparing point estimates from a handful of runs is old, and the recommendation against it is old too. Henderson et al. showed in 2018 that variance in deep reinforcement learning can swamp algorithm differences, and argued for repeated runs and reporting discipline.[5] Agarwal et al. pushed the same argument into methodology: interval estimates instead of point estimates, performance profiles for qualitative comparison, robust aggregates such as the interquartile mean, and a public library to compute them.[6] Our setting differs in ways that matter for the protocol. Their runs are independent, seeded training runs on a suite of tasks; our runs share one GPU and one inference server, game outcomes are long-horizon trajectories whose seeds cannot be fixed, and the scoring environment that matters most (the competition rerun) cannot be repeated on demand. Sections 3 and 4 describe how the protocol adapts their recommendations to that setting. We claim no novelty for the idea of repeating experiments; the contribution is the concrete evaluation protocol and the resource accounting, validated on this benchmark.

What wins here. The papers that took this competition's paper awards in 2024 and 2025 introduced new learning or search mechanisms: induction plus transduction,[16] test-time training,[17] latent program search,[18] tiny recursive networks,[13] evolutionary program synthesis,[14] and compression-driven learning without pretraining.[15] Each reports substantial score gains on a small number of runs. How much of such a gain survives a repeat is a question the wider community has only recently begun to ask, and 2026 work on agent reproducibility has taken it up directly: an intensive replication study of coding agents across 584 runs found run-to-run variation exceeding the differences between agent-model pairings,[19] and a reproducibility standard for agent research makes rollout records, not reported scores, the unit of publication.[20] Our contribution is not the observation that reruns vary; it is a concrete evaluation protocol and request-level resource accounting for this competition's particular constraints, demonstrated on our own entry.

ARC Prize and its tracks. ARC Prize 2026 runs three tracks: a static reasoning benchmark (ARC-AGI-2), the interactive benchmark we compete on (ARC-AGI-3), and a paper prize for work that advances the understanding of how to reach strong ARC performance.[1] The paper track requires a write-up linked to a Kaggle submission, with the submission's score feeding one of six equally weighted rubric categories; the submission need not score highly.[1][2] Our entry links a mid-pack submission at 30.21 and the analyses below.

Scoring. ARC-AGI-3 scores relative human action efficiency (RHAE). For each completed level, the agent earns the square of the ratio of human baseline actions to agent actions, capped at 1.15, and the level scores are averaged with the level index as weight; the game score is additionally capped at the weighted fraction of levels completed, and the benchmark score is the mean over games.[3][4] Section 7 reads our results through that formula rather than through informal accounts of difficulty.

The stack we build on. The Duck harness from Tufa Labs won the first ARC-AGI-3 milestone.[9] Its design, carried into the milestone-2 solutions, is a tool-using solver: the language model reasons in text, writes Python in a sandbox to inspect the board and probe mechanics, and executes game actions through the same interface. Franzen's milestone-2 solution is the base of our entry, and his write-up documents the changes he found most impactful: a larger context window, exposing animations and difference images, making UNDO available, and a scheduler that spends more compute on games showing promise. He also reports what did not work, including repeated-state guards and a death ledger, which matches what we found.[7]

Memory and exploration. Two recent papers frame the problem differently from configuration tuning. PRO-LONG keeps a structured interaction log and lets the agent search it programmatically, reporting an average gain of 18.0 percentage points over a base coding agent on the ARC-AGI-3 public set with frontier models.[10] A 2026 study of the public set finds that every one of the 25 public games can be reached by non-intelligent strategies, so public-set scores cannot discriminate intelligent exploration; the private evaluation is the real test.[11] Both results shaped how we read our own numbers: the public set is a weak proxy, and the lever with the strongest published evidence is memory, not prompt engineering.

Variance reports in this competition. Variance is not a private discovery. Lohit's milestone-2 write-up reports the same notebook submitted three times scoring 5.02, 5.50 and 6.42, and a practice run of 37 turning into 9.98 on the leaderboard.[8] He describes the response we arrived at independently: compare runs at equal generated tokens rather than equal wall time, prefer full-length runs, and read transcripts instead of trusting scores. What the public record lacks, and what this paper tries to supply, is a stated protocol with decision rules rather than a warning.

3 Setup

Hardware and stack. One NVIDIA RTX PRO 6000 (97,887 MiB reported). Qwen3.8-Flash-Next in Intel's W4A16 AutoRound quantization, with a separate INT4 MTP draft model for speculative decoding, served by SGLang Pennyroyal v2.5.3 on the pinned Python 3.12 image, offline. The harness runs with a 131,072-token window, a 12,288-token output cap, KV cache in FP8 E4M3, context evictions that drain roughly 58 Ki tokens at a time (`ARC3_CONTEXT_DRAIN_TOKENS`, a token count and not a byte size), and admission limited to ten concurrent game streams. These values match Franzen's open milestone-2 configuration; we verified them against his write-up and against the source of the submitted notebook (`docs/rerun-config-audit.md` in the open-source tree).

Our patch. On top of the published source bundle we apply our own harness patch, 16 files and roughly 7,136 added lines, which adds 94 environment switches. All switches default to off unless a run turns them on; a run's effective configuration is the switch set it enables, and only that set appears in the tables. The switches cover history trimming, prompt variants, vision helpers, persistent Python functions, scheduling, and the guards discussed in Section 6. Applying the patch to the published bundle is verified mechanically; the patch hash and the bundle hash are recorded with the release. The statements in this paper about third-party code are limited to what we changed; attribution and licenses for inherited material are listed in `THIRD_PARTY_NOTICES` in the release, and our MIT-0 license covers only our own additions.

Batch runner. Experiments run under a small supervisor that gives each arm its own worker process, writes native artifacts only, never imputes a score for a failed game, and stops the batch when a safety bound trips. An arm is a `(profile, budget, environment overrides)` tuple; because the overlay is applied per worker, one run can compare several arms against one server and one game set. A cold server start costs about 25 minutes of the budget, so the server is restarted only when a serving knob changes.

Game sets and budgets. Screens use the first ten games of the sorted public list, 2000 seconds per game, one pass, ten streams. Calibration runs use 25 games; the per-game ceiling is 2900 seconds except for a4, which raises it to 7200. Both sets are frozen and shared across the arms of a run. For reference, the competition rerun gives each of 110 games a much larger budget, a 532-minute wall-clock ceiling with the same ten streams.

The protocol, and when we adopted it. The bracketing protocol described here was adopted part-way through the project, and this section distinguishes what followed it from what predates it. A fully bracketed screen contains a baseline arm, one or more candidate arms, and, executed last, an identical repeat of the baseline; the arms run in a fixed order against one server, so we call the first and last arms same-session control arms rather than concurrent runs, and we report the gap between them as the observed same-configuration variation of that session, not as a time trend. Two screens (E5b and E6) follow this design in full. The E5 screen predates it and lost its last arm to an infrastructure failure, so its candidate reading has no end control; the 25-game calibration runs (a3, a4, a5) are single-execution runs of a different kind and are used for latency observation and, in one case, a nominally matched comparison. A candidate is read against both the baseline beside it and the end control: if its margin over the end control is smaller than the session's observed variation, the comparison fails the screening heuristic, and we record it as unresolved. The heuristic is a bookkeeping rule for our own decisions, not a statistical test; it was applied retrospectively across this paper's record rather than prespecified for the early sessions, and we do not treat any of its thresholds as transferable to other settings. Batches record per-request timings from the harness's event log and per-game scores from the native metrics, so every mean in this paper can be recomputed from artifacts rather than trusted from a summary line; `tools/build_numbers.py` performs that recomputation.

Run comparability. Where two runs are compared for an intervention effect, we state exactly which keys differ. The pair used for the timeout intervention (runs a3 and a5) differs in four settings, which are listed in Table 2; the other three are budget companions of the first one and were changed with it deliberately. The pair used for latency anatomy (a4 and a5) differs in more, including the per-game ceiling and the number of sessions competing for the ten admission slots, and is used for observation only. Appendix A lists the traps we hit while building this out.

4 Same-configuration variability

We did not set out to study variance. It found us. In our first six-arm screen we added a repeat of the baseline as an afterthought, mostly because the batch clock allowed it, and the repeat came back 3.3 points above the arm it duplicated. Table 1 collects every same-configuration comparison we have.

Table 1. Identical configurations, measured more than once. Local means are over the frozen ten-game set, 2000 seconds per game. Sources: `runs/e5_behavior`, `runs/e5b`, `runs/e6`, `runs/e7`, Kaggle submission list.

Setting	Run	Mean score	Levels	Actions
baseline	E5 screen	12.91	17/73	552
baseline	E5b screen, four days later	14.90	21/73	635
baseline repeat (same session as E5b, run last)	E5b screen	18.22	20/73	655
baseline	E6 screen, next day	11.52	17/73	631
baseline repeat (same session as E6, run last)	E6 screen	14.71	20/73	625
baseline, arm 1 of 6	E7 matrix	12.71	20/73	617
baseline, arm 3 of 6	E7 matrix	13.53	20/73	656
baseline, arm 5 of 6	E7 matrix	12.30	17/73	664
submitted build v2, submission 1	Kaggle LB	29.28	-	-
submitted build v2, submission 2	Kaggle LB	24.38	-	-
submitted build v2, submission 3	Kaggle LB	29.41	-	-
submitted build v2, submission 4	Kaggle LB	29.60	-	-

Five spreads come out of this table, and the fifth is the one that matters most. Within one session, the same configuration differed by 3.33 points between the first and last arms of E5b, about four and a half hours apart, and by 3.20 points across the same bracketing arms in E6 the next day. In a third session, the alternating matrix E7 ran three identical baseline arms interleaved with candidates, and those three samples span only 1.23 points (12.71, 13.53, 12.30). Across sessions, the first-arm baseline differed by 1.99 points, 12.91 against 14.90. Across leaderboard reruns, one code version spanned 5.22 points, with a mean of 28.17 and a median of 29.35 over the four samples. Figure 1 shows both panels.

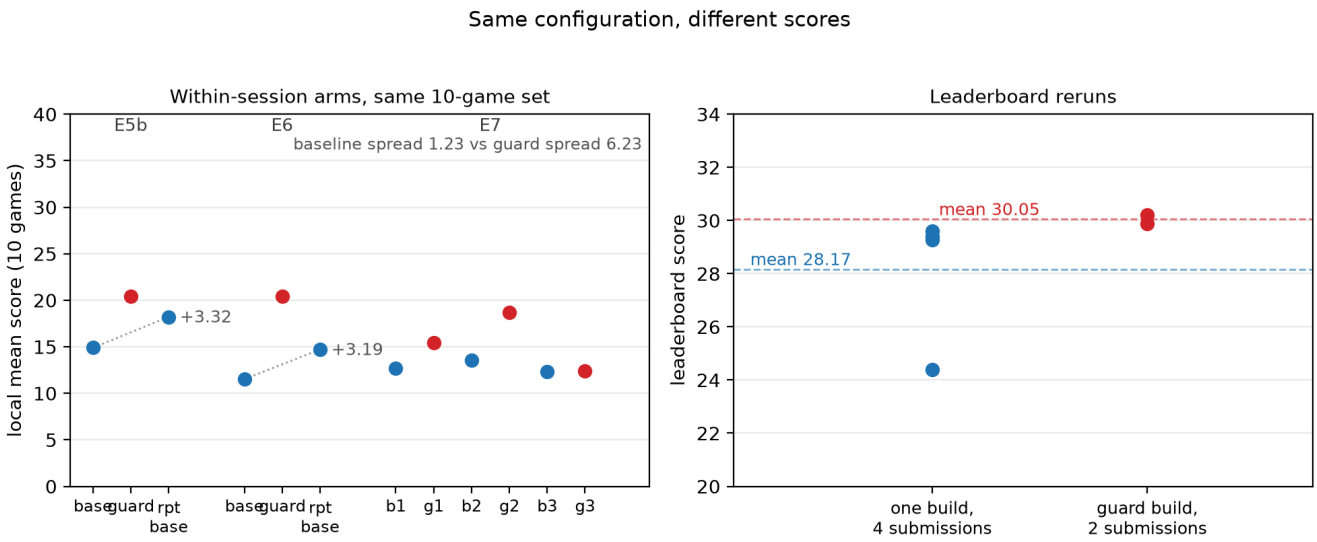


Figure 1. Same-configuration variation. Left panel: arms of the three same-session screens on the frozen ten-game set, blue for baselines and red for guard arms, with the dotted line marking each bracketed screen's end-control gap. Right panel: the six leaderboard submissions. Four points belong to one build and two to the other; each build's own points differ by more than most candidate changes could hope to deliver.

We treat the within-session gaps (3.33, 3.20, 1.23) as the observed same-configuration differences of their sessions, not as a statistical noise floor, and the fact that they vary by a factor of almost three under one benchmark and one stack is the point. Three measurements still cannot estimate a distribution, a confidence interval, or a detection threshold; what they can do is falsify the assumption that same-configuration runs agree, which is the assumption most single-run comparisons rest on. The leaderboard spread, four samples of one build, says the same thing at the scale that decides prizes. None of these numbers transfers to a different setting, and none should be quoted as a constant. A second caveat: a difference between two same-configuration arms is not evidence of drift over time. Two runs can differ through sampling, through trajectory divergence, or through environment changes; separating those would need more repeats than we could afford, and the E7 session, whose baselines do not rise monotonically, is a reminder that the first two sessions' end-arm bump need not be a trend.

The E7 session carries a second lesson that reaches further than the baselines. Variance is not symmetric across configurations. In that one session the three baseline arms spread within 1.23 points, sample standard deviation 0.63, while the three guard arms spread 6.23 points, sample standard deviation 3.12, and their three paired differences had a sample standard deviation of 2.50, with a range of 5.00. A team that measures only the baseline's variability would understate how uncertain the candidate's own result is: baseline variability alone can be insufficient for screening an intervention, because the candidate configuration may be substantially more variable than the baseline it is compared with. Whether that asymmetry reflects a trajectory-altering mechanism that helps only in some runs, or sampling and order effects amplified in longer runs, cannot be decided by three pairs. What the data supports is the caution itself.

These are not artifacts of our harness. Lohit's milestone-2 write-up reports the same notebook submitted three times scoring 5.02, 5.50 and 6.42, and a practice score of 37 turning into 9.98 on the leaderboard.[8] In the milestone comparison thread, participants swap single-run before-and-after numbers and hedge them as guesses, which is the epistemic situation this benchmark forces on anyone without a spare GPU week.[12]

Two practices follow, and we adopted both for every number in this paper.

First, a screen needs its own control. Absolute local scores move between sessions by about two points and vary within a session, so a candidate is read only against an arm that ran in the same session, over the same frozen game list, behind the same server.

Second, comparisons need a bracketing control. Our protocol places an identical copy of the baseline at the end of the batch, and a candidate whose margin over the end control is smaller than the session's observed variation is reported as unresolved. In the E5b screen that variation was 3.33 points and in E6 it was 3.20, which is why several results whose margins fall inside that band carry the label "inconclusive" instead of "gain".

A third practice is cheaper: read transcripts, not just means. A screen can lose points because one game collapsed, and only the transcript tells you whether that was a bad idea or a bad roll of the sampling dice. Section 6 uses that reading twice, once for a game the guard solved in all three runs, and once for the same game solved by a control arm in a fourth.

5 Where the compute goes

If the observed variation dictates what can be measured, the compute budget dictates what can be tried. The competition rerun gives a run nine hours of wall clock, ten concurrent game streams, and a per-game ceiling of 532 minutes. The question that matters for harness work is not how much compute there is but where it lands.

Each worker writes a request-level event log and the serving process writes a per-request summary line. Merging those gives Table 2, which puts the stock 120-second analyzer timeout next to a doubled one.

Table 2. Request-level accounting, 120 s vs 240 s analyzer timeout. Runs a4 and a5 also differ in their per-game ceilings (7200 s against 2900 s) and in how many sessions compete for the ten admission slots (25 against 10), so this table is used for latency anatomy only. The score comparison in the last row uses runs a3 and a5, which differ in exactly four settings: `analyzer_timeout_seconds` 120 to 240, `graceful_stop_seconds` 130 to 260, `worker_overhead_seconds` 250 to 500, and `global_limit_seconds` 13000 to 13500. The last three are budget companions of the first. Sources: `runs/calib_a4/.../request-events.jsonl`, `runs/calib_a5/.../request-events.jsonl`, both `serve.log`, and the two runs' configuration cells.

	timeout 120 s (run a4)	timeout 240 s (run a5)
requests	1500	1453
median duration	34.0 s	28.3 s
90th percentile	120.1 s	114.5 s
maximum	120.2 s	240.1 s
requests terminated at the cap	225 (15.0% of requests)	38 (2.6%)
requests that exceeded 120 s	225 (15.0%)	135 (9.3%)
total request time	19.98 h	19.16 h
errors reported per game, sum	235	53
mean score, 25 games (a3 at 120 s vs a5 at 240 s, nominally matched)	27.40	24.20

The pile-up at the cap is the tell. The 90th percentile in the first run sits at 120.1 seconds, and 225 requests end there. The timeout-terminated requests alone account for about 7.5 hours of request duration, 37.5% of the 19.98-hour total, and they dominate the error difference between these two runs, 235 against 53. Two readings of that share are tempting and both are too strong. Request duration is not GPU time: the server interleaves up to ten streams, and a request that occupies the queue is not a request that occupies the card. And a terminated generation is not automatically a worthless one: part of what a long generation produced may have shaped the agent's next steps, and our logs do not let us separate valuable long reasoning from wasted long reasoning. What the number does establish is that a third of the analyzer's cumulative request budget went to requests the client itself cut off, which makes the timeout a first-order term in resource planning, whatever its score effect.

Its score effect remains unresolved, and the observed direction is a drop: 24.20 against 27.40 for the nominally matched run with the shorter cap. Raising the cap moves the discarded trade into completed ones: at 240 seconds the cap terminates 38 requests instead of 225, and the reported error count in that pair falls from 210 to 53, although the count is itself a multi-key comparison, since the three budget companions changed with the cap and can affect execution paths, truncation behaviour and real work. Because each configuration ran once and the intervention included the companion keys, the comparison can identify neither a stable beneficial nor a stable harmful score effect.

The generated-token distributions explain why the tail is where the work is. Over both runs the median request generates about 1,300 tokens, the 90th percentile generates 4,265 to 5,099, and the largest single generation reaches the 12,288-token ceiling in the second run, where long generations are no longer cut off at 120 seconds. Roughly 22% of requests exceed 3,000 tokens in both runs. We cannot show that the longer generations bought score; we can show that they absorbed most of the tail, and that giving them more room did not change the outcome we can measure.

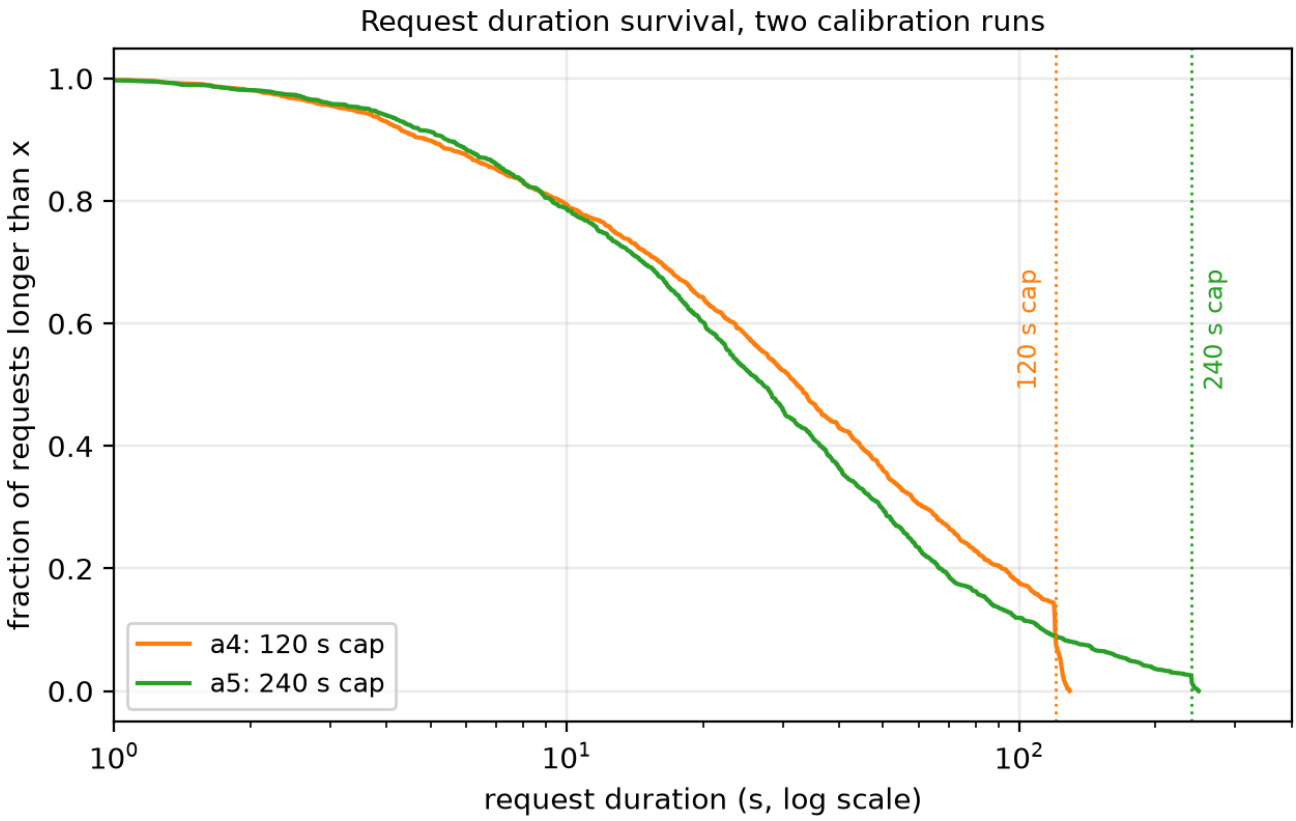


Figure 2. Request-duration survival, log scale. The 120-second cap censors 15.0% of the first run's 1,500 requests; at 240 seconds only 38 of the second run's 1,453 requests reach the cap.

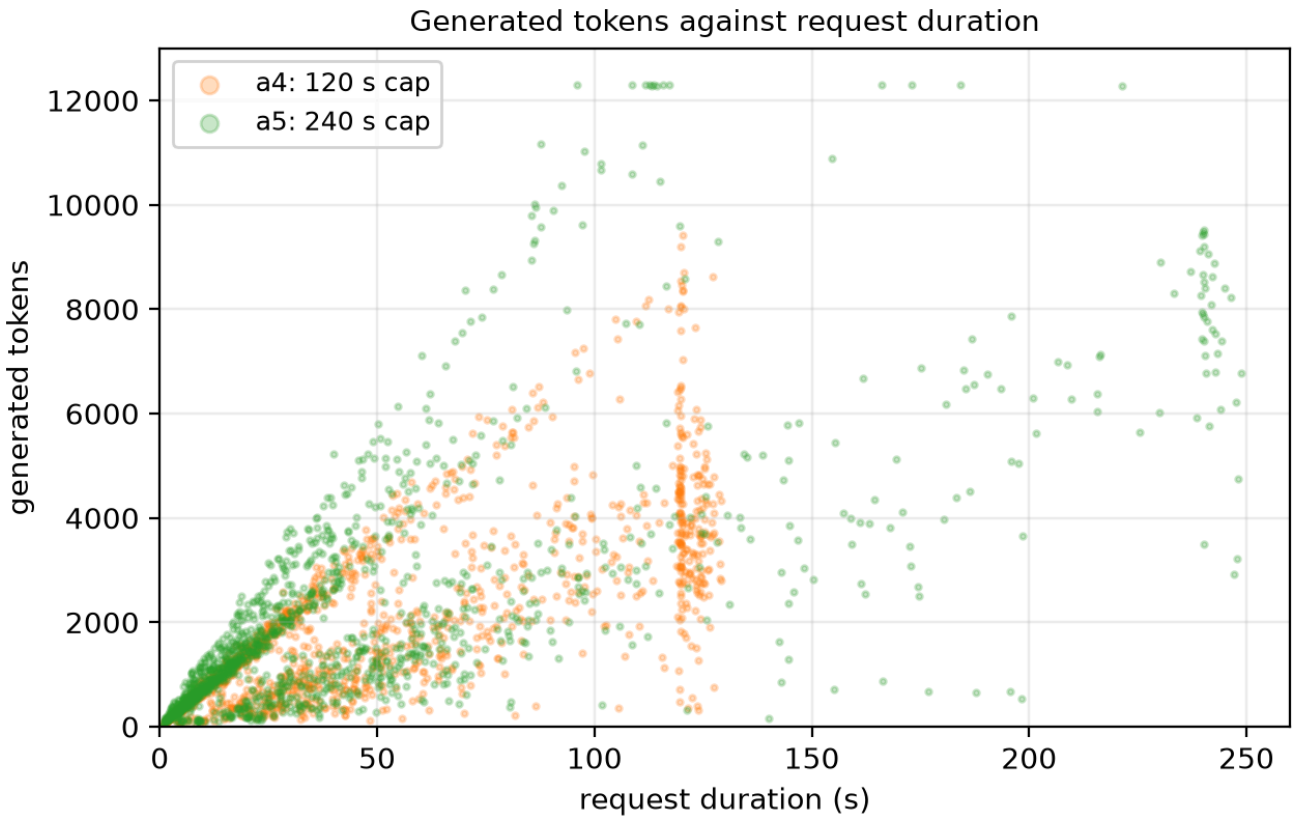


Figure 3. Generated tokens against request duration. The longest generations live in the censored tail; raising the cap lets them finish instead of truncating them.

One more line from the same logs, with a caveat attached. Queue wait behind other streams is smaller than generation time in these two runs: the median request waits 0.03 seconds before the server starts it, the 90th percentile waits 11 seconds, and the worst case is 108 seconds. That is not the same as saying queueing is negligible, and a different workload, for example many short requests, would move the bottleneck; in the logs we have, generation time dominates.

6 Ablations: what transferred and what did not

Rows measured under the full protocol (E5b, E6 and the E7 matrix) come from sessions containing control arms as described in Section 3; the remaining rows carry their own designs and are labeled accordingly. "Local" means the frozen ten-game set at 2000 seconds per game unless noted; "LB" means the public leaderboard. The outcome column uses four labels and no others: **inconclusive** when the estimate cannot be separated from same-configuration variation, **promising locally, unconfirmed** when the local margin passes the screening heuristic but the leaderboard provides no supporting sample, **no observed improvement** when the local estimate is at or below zero, and **descriptive** for rows where no controlled comparison exists. We keep the three experiment families, harness ablations, throughput, and budget, separate, because they answer different questions; the earlier draft of this paper counted them together, which was wrong.

Table 3. Changes we tested, and what the evidence supports. Sources: the run directories named in the local-delta column and the submission list in Table 1.

Change	Family	Local estimate	LB evidence	Outcome
Repeat and death guards (`ARC3_NOOP_REPEAT_GUARD`, `ARC3_DEATH_LEDGER`, `ARC3_DEATH_REPEAT_GUARD`)	harness ablation	+9.12 (E5), +5.54 (E5b), +8.91 (E6) vs the arm's first baseline; margins over the paired control +2.21 (E5b), +5.71 (E6), +2.72/+5.13/+0.13 (E7 pairs)	two submissions of this build, 57147727 at 29.89 (10-13) and 57174905 at 30.21 (10-14), against 29.60 and a 28.17 mean for the older build; unpaired	inconclusive
History turn drain 30 to 60 (`ARC3_HISTORY_TURN_DRAIN`)	harness ablation	+6.02 (E5b), +5.19 (E6) vs first baseline; margins over the end control +2.7 and +2.0	kernel build prepared; not submitted (the daily submission on 10-14 went to the guard rerun)	inconclusive
Level inventory on new levels (`ARC3_LEVEL_INVENTORY`)	harness ablation	-1.30 (E5b)	not submitted	no observed improvement
Persistent memory sections, slim (`ARC3_MEMORY_SECTIONS=slim`)	harness ablation	+0.67 (E5b)	not submitted	inconclusive
Admission 12 instead of 10	throughput	+12.4% actions/s, below the run's own +15% bar	not submitted	no observed improvement
Analyzer timeout 120 to 240 seconds	budget	-3.20 (a3 vs a5, 25 games, nominally matched)	not submitted	inconclusive
Scheduler gate: 25 games sharing 10 slots	throughput	no starvation observed, -2.04 vs the ungated run	not submitted	no observed improvement
More time per game (practice budget, 1500 s to 2000 s)	budget	6.8 vs 12.9 to 18.2 mean, different sets and runs	-	descriptive; confounded with set and session

Table 3a. Submission identifiers behind the leaderboard rows. All six are competition submissions of the same kernel, `zgyyyl/arc-agi-3-daniel-m2-baseline`; the competition allows one scored submission per day, so no two rows on the same UTC date exist.

date (UTC)	submission	build	result
2026-10-06	56882441	base build: Franzen milestone-2 solution plus our patch, every switch off	29.28
2026-10-10	57037255	identical code to the above	24.38
2026-10-11	57061252	identical code	29.41
2026-10-12	57100038	identical code	29.60
2026-10-13	57147727	base build plus the three guard switches (kernel version 3)	29.89
2026-10-14	57174905	byte-identical to 57147727, re-submitted for a second sample	30.21

The guard result deserves the detail, because it is the clearest case of a local estimate that the leaderboard could not resolve, and because the mechanism is sound on inspection. The guards watch the board state outside the border of the play area. When an action leaves that state bit-identical, they record the triple of (level, state, action) and refuse to execute the same action again in the same state. A refused action arms a single slot: if the model immediately re-issues it, it executes, which keeps the guard from sealing off a hypothesis it got wrong.

Across four sessions the guard arm ran six times. Two of those runs belong to fully bracketed screens (E5b, E6) and three to the alternating E7 matrix; the E5 run has no control. Read as first-baseline differences, the six runs beat their control arm by +9.12 (E5), +5.54 (E5b), +8.91 (E6) and +2.72, +5.13, +0.13 (E7 pairs). Five of the six comparisons carry a screening condition, two of the bracketed kind and three of the alternating kind, and the two designs are not interchangeable, so we report them separately: one of two bracketed comparisons and two of three alternating comparisons satisfied their respective exploratory screening conditions. These conditions flag observations for further study, not statistical significance, and the guard runs' own dispersion, far wider than the baselines' in the same session, prevents a reliable estimate of the guard effect. The per-game tally over all sixty comparisons (six guard runs against the designated control arm of their screen; a tie is a difference below 0.01 points) is 22 wins, 15 losses and 23 ties, though those sixty reuse the same ten games and are not independent units. On ft09, guards have reached 100 points in three of their six runs, against two such solves in thirteen non-guard arms across the four sessions; in E7 the game gave a smaller guard gain in one pair (28.57 to 45.27) and ties in the other two, so the clean three-for-three ft09 story of the earlier sessions does not extend. A second game, cd82, improved with the guards in four of the five paired comparisons (largest +44.7). With ten games to a screen, one flipped game moves the mean by up to ten points, which is the trap this protocol exists to catch.

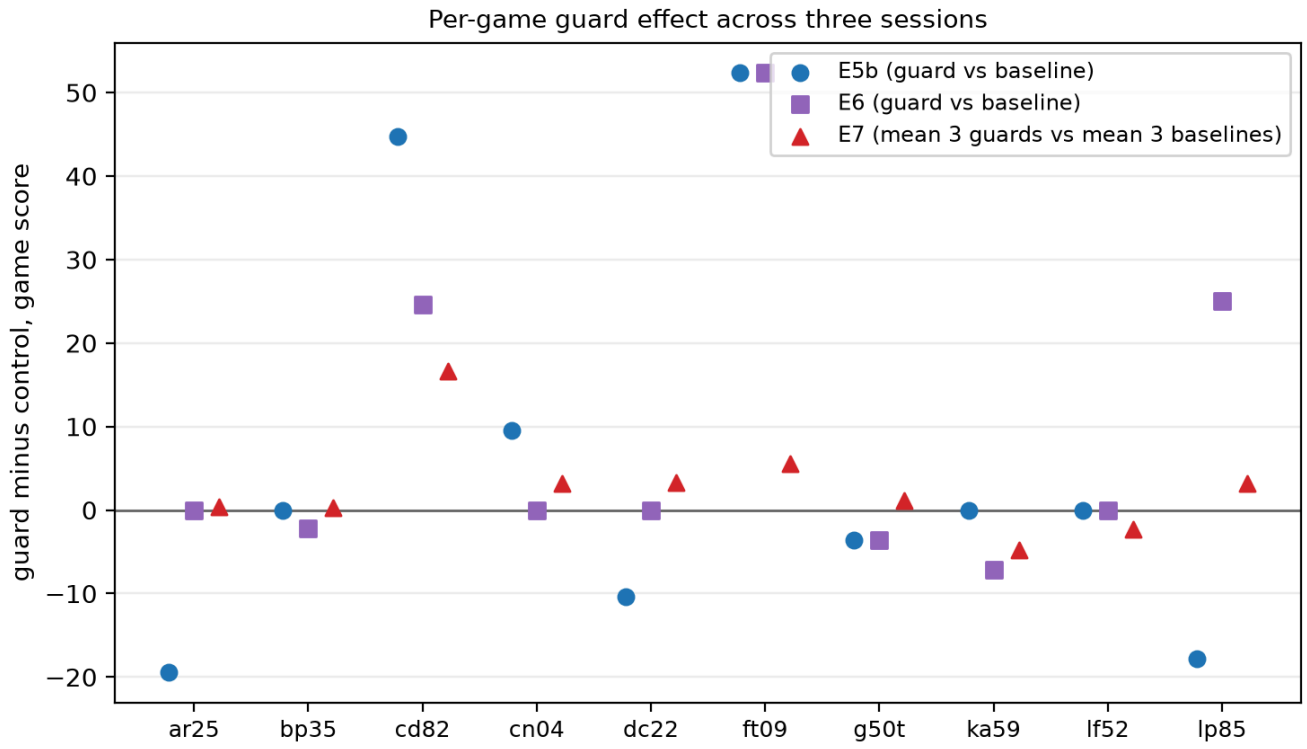


Figure 4. Per-game guard deltas. Each dot is one game's score difference against its same-session control; the gains concentrate in a different game in each session, which is why the ten-game mean moves with them.

On the leaderboard the guard build was submitted twice and scored 29.89 and 30.21 (mean 30.05) against the build it replaces, whose four samples average 28.17. Both readings are descriptive and unpaired: +1.88 against the older build's mean, +0.61 against its best sample, and the guard build's own two samples differ by 0.32. None of that is an intervention estimate. Franzen reports the same kind of result from the other side of the competition: he explored repeated-state guards, remembered no-op or fatal-action guards, and a death

ledger, and left them out of his submission for lack of clear benefit.[7] Two independent attempts, one of them by the milestone winner, found nothing either could confirm.

The rows that needed no new measurement are worth naming for the same reason. The competition's largest published jumps come from retaining information: Lohit switched to an FP8 KV cache and spent the freed memory on history, and the change "took the score from 14.49 to 22.53"; the milestone comparison thread describes the same trade at the same magnitude.[8][12] Our own calibration agrees in direction, since the base configuration we run already spends its memory on a 131k-token window and a 58k drain, and our attempts to go further by adding structure (memory sections, level inventories) did nothing measurable. The difference between what those teams did and what we tried is the mechanism, not the goal: they bought more room for the agent's own history, and we added more instructions about what to put in it.

Evidence strength. The rows of this paper rest on different amounts of evidence, and Table 4 states it rather than leaving the reader to sort it out. Nothing in this paper rises to the level of a repeated controlled comparison on the target environment; the leaderboard samples are descriptively useful and statistically weak.

Table 4. Evidence behind each finding.

Finding	Evidence type	n	Strength
Same-configuration local differences, 3.33 / 3.20 / 1.23	bracketing and alternating repeats in three sessions	2, 2 and 3 samples	observation
Same-code leaderboard spread, 5.22	repeated submissions, same build	4 samples	observation
Guard build leaderboard samples, 29.89 and 30.21	re-submissions of one build, unpaired against another build's four samples	2 vs 4	descriptive
Timeout share of request duration, 37.5%	observational logs, two runs	2 runs, 2,953 request events, nested within runs and games rather than independent	measurement
Timeout intervention, no demonstrated score change	nominally matched runs, single execution each	1 vs 1	inconclusive
Guard local estimates: first-baseline deltas +5.5 to +9.1; E7 paired deltas +0.1 to +5.1	four screens; five comparable readings (two bracketed, three alternating)	6 arm runs, 10 games each	one of two bracketed and two of three alternating comparisons satisfied their screening conditions; magnitude unstable
Guard leaderboard estimate	one re-submission of one build against samples of another, unpaired	2 completed, vs 4	descriptive
ft09 guard solves, 3 of 6 vs 2 of 13	per-game records across four screens	19 arm runs	suggestive
External reports of variance	other teams' write-ups	2 sources	external corroboration

What the protocol changes: a retrospective decision audit. Decision rules are only worth stating if they change decisions, so we replayed our own record under three rules: a naive read that compares a candidate with the first baseline of its session, the bracketing rule that additionally requires the margin over

the end control to exceed the session's observed variation, and the repeated-control view once E6 gave us a second controlled comparison.

Candidate reading	Naive: first baseline	Bracketing rule	Repeated-control view
Guards, E5	+9.12, keep	no control in that session	-
Guards, E5b	+5.54, keep	+2.21 vs control, below 3.33, unresolved	-
Guards, E6	+8.91, keep	+5.71 vs control, above 3.20, passes	guard group: 1 of 2 bracketed and 2 of 3 alternating comparisons satisfied their screening conditions; magnitude unstable
Guards, E7 pair 1	+2.72 vs its control, keep	above E7's 1.23 band, passes	
Guards, E7 pair 2	+5.13, keep	above 1.23, passes	
Guards, E7 pair 3	+0.13, keep	below 1.23, unresolved	
Drain 30 to 60, E5b	+6.02, keep	+2.69 vs control, below 3.33, unresolved	0 of 2 controlled comparisons pass
Drain 30 to 60, E6	+5.19, keep	+2.00 vs control, below 3.20, unresolved	
Combo, E6	+4.10, keep	+0.90 vs control, unresolved	-
Inventory, E5b	-1.30, drop	-1.30 vs baseline, -4.63 vs control, drop	-
Memory slim, E5b	+0.67, keep	+0.67 vs baseline, -2.66 vs control, unresolved	-

Ten of the eleven readings are positive, and nine of those can be assessed under a screening rule (E5 has no control). Six of the nine fail their screening condition and three pass, all of them guard readings. The audit cannot show that the protocol produces better decisions, because we have no ground truth for the true effect of any candidate; what it shows is that the protocol changes which claims survive contact with the data, and that the guard's instability, invisible under a naive read, becomes explicit. Leaderboard observations remained insufficient to validate or reject the guard effect, and most other candidates were not evaluated on the leaderboard.

7 Why the score responds this way

The scoring rule shapes everything above, and it is worth stating precisely. ARC-AGI-3 scores relative human action efficiency.[3][4] For a level `l` in a game `e`, with the agent taking `a` actions and the human baseline, an upper-median of first-time human players, taking `h`, the level score is the squared ratio `(h / a)^2`, capped at 1.15 so that a single shortcut cannot dominate a game. A game's score is the weighted average of its level scores, with the level number as the weight, so level five counts five times level one, and it is additionally capped at the weighted fraction of levels completed. The benchmark score is the mean over games. Three consequences follow.

Depth dominates, twice over. An unsolved level scores zero and also lowers the game's cap: a game with five levels where four are cleared can score at most 10/15, or 66.7%, however well those four were played. One

game that flips from unsolved to completed is therefore worth up to ten points on a ten-game mean, more than any efficiency change can deliver, and the level weighting means the late levels carry the most weight per level. This is why screens of ten games are this noisy, and why the same-configuration variation of Section 4 sits where it does.

Efficiency is penalized quadratically, but in the runs we inspected it is not the binding constraint. On levels our agent does solve, it is already at or below the human action count. One arm's trajectory on ar25 used 17, 17, 43, 30 and 33 actions against human baselines of 32, 50, 75, 37 and 89 for the five levels it cleared, which earns full or capped credit on each. Once the agent understands a level, its actions are few; the cost of not understanding it is bounded only by the budget, since the agent keeps acting without progress until the level's limit stops it, and the square turns a twofold excess into a quarter of the credit.

Compute has diminishing returns in the place where it is easiest to spend. In our runs, longer deliberation per action did not repair a wrong world model: doubling the analyzer request timeout in the Section 5 experiment changed the error profile (210 to 53 in the matched pair) without a score change we could measure. Whether longer deliberation helps in other settings is untested here. Longer context and better memory are a different matter, and they are the interventions with replicated published gains on this benchmark.[8][10] If we were continuing rather than writing up, we would stop re-tuning switches and start building the searchable interaction log that PRO-LONG describes, which a small model on one GPU needs even more than a frontier one.

The last piece is why local screens are weak proxies for the leaderboard, beyond variance. The study of the public set cited above shows that the entire game set is reachable with non-intelligent strategies, and that the private evaluation is the only real test.[11] Lohit watched a 37-point practice run turn into 9.98.[8] Our two best builds sit within 0.6 points of each other on the leaderboard, 29.60 and 30.21, while their local readings differ by more than five points. Local measurement is useful for finding regressions and for tearing down broken mechanisms, and this paper shows it can be made disciplined enough to do that; it is not a ranking device, and treating it as one is the specific failure mode this paper documents.

8 Limitations

We report from one machine, one model, and one competition entry, and several reviewer questions have no good answer in the current data.

The samples are small in the way that matters. Same-configuration variation rests on three sessions of control arms (two, two and three samples) and four leaderboard samples of one build; none of that estimates a distribution, and no number in this paper should be read as a detection threshold. The guard's leaderboard evidence is two submissions of one build against four of another, unpaired, which bounds the effect only above roughly five points; a real effect of one or two points cannot be excluded by anything we ran. Four screens of ten games each are enough to catch a game-sized flip and not enough to estimate an effect size, which is why the guard row reads inconclusive and not "no effect" or "works".

The comparisons are not all clean. The a3 and a5 runs differ in four settings, all named in Table 2, of which one is the intervention and three are its budget companions; that is close to a single-factor comparison and not identical to one. The a4 and a5 runs differ in more than the timeout and are used only for observation. Where we report a local estimate against a same-session control, the arm order is fixed by the batch, so an order effect and an intervention effect cannot be separated without a reversed-order replication, which we did not run: in the E7 matrix, each candidate arm always follows the arm it is compared with.

Selection is a real risk in a paper about measurement discipline. We ran many small screens over the competition, several aborted by infrastructure failures, and the tables here report the runs that completed with usable artifacts. Appendix C lists every experiment we started, its state, and why it entered or left the analysis. Decisions about what to report were made after seeing the artifacts; we have tried to make that safe by reporting estimates against controls and by keeping negative and inconclusive rows in the table rather than dropping them.

Finally, local measurement inherits the public-versus-private gap. The public set is reachable by non-intelligent strategies,[11] the private evaluation is the only test that counts for the prize, and no local number in this paper can be read as a leaderboard prediction. What the protocol can do is keep a team from shipping noise and from abandoning a mechanism for the wrong reason.

9 Conclusion

The practical content of this paper is a protocol and a table. The protocol: keep the game set frozen, bracket every batch with an identical control arm, report the gap between controls as the session's observed variation, read candidates against the control rather than against the previous session, and let estimates that do not clear that variation carry the label that says so. The table: of the changes we could test, none is resolved by leaderboard evidence, and the two interventions with published, replicated gains on this benchmark, both of which buy the agent more room for its own history, are the two we could not fully implement here.

For anyone entering the next round of this competition, the advice is unglamorous. Spend your first GPU day measuring your own variation, because the number you find will change what you believe about every change you are about to make. Then spend your effort on what the agent remembers, not on how cleverly it is told to behave. The rest, as one participant put it while comparing the milestone winners' approaches, is inference engineering.[12]

Reproducibility and AI assistance

The harness patch, the notebooks, the analysis scripts and the raw artifacts behind every table are released under MIT-0 for our own code at `github.com/guoyi12311125-netizen/arc3-repeat-controlled-eval`, with a Kaggle dataset mirror at `kaggle.com/datasets/zgyyyl/arc3-paper-release`; inherited components keep their upstream licenses, itemised in `THIRD_PARTY_NOTICES` in the release.[1] The competition submission linked to this paper is 57174905, a re-submission of the guard build described in Section 6 with a public score of 30.21; its notebook version and the effective configuration are recorded in the release manifest. The patch is verified to apply cleanly to the published source bundle, with both hashes recorded in the release. Every number in this paper is recomputed from stored artifacts by `tools/build_numbers.py`; the run directories, the submission identifiers, and the analysis scripts are listed in the release README. Appendix A documents the protocol, including the traps that cost us days.

Generative AI tools, including Claude Code and a Hermes-based agent, were used to assist with implementation, experiment orchestration, log analysis, data-processing scripts, and manuscript drafting and editing. The author directed the research questions, selected the experimental comparisons, evaluated the resulting evidence, and determined the conclusions presented here. AI-generated outputs were not treated as independent experimental evidence: every reported number derives from recorded experiment artifacts through the accompanying analysis scripts. The author takes responsibility for the accuracy of the

manuscript, the reproducibility of the reported analyses, and the attribution of third-party code and methods.

Acknowledgements

The Duck harness and the TAAF framework come from Tufa Labs and won the first ARC-AGI-3 milestone; the base notebook is Daniel Franzen's milestone-2 solution. Their write-ups, including the negative results, shaped our testing more than any single feature. We also thank the participants whose discussion threads supplied the variance reports quoted above, and the ARC Prize team for a benchmark that makes measurement this interesting.

References

[1] ARC Prize Foundation. ARC Prize 2026, Paper Prize. arcprize.org/competitions/2026/paper [2] ARC Prize 2026 Paper Track, Kaggle competition page. kaggle.com/competitions/arc-prize-2026-paper-track [3] ARC Prize Foundation. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. arXiv:2603.24621, March 2026. [4] ARC-AGI-3 Scoring Methodology. docs.arcprize.org/methodology [5] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., Meger, D. Deep Reinforcement Learning that Matters. AAAI 2018. arXiv:1709.06560. [6] Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., Bellemare, M. G. Deep Reinforcement Learning at the Edge of the Statistical Precipice. NeurIPS 2021, Outstanding Paper. arXiv:2108.13264. [7] Franzen, D. ARC-AGI-3 Milestone 2: Building on the Duck Harness. GitHub da-fr/arc-agi-3-solution, WRITEUP.md. [8] Lohit, S. (rellik13). ARC-AGI-3 Milestone 2 write-up. GitHub LohitSiriki/arc-agi-3-milestone2-solution, WRITEUP.md. [9] Tufa Labs. duck-harness. GitHub Tufalabs/duck-harness. [10] PRO-LONG: Programmatic Memory Enables Long-Horizon Reasoning. arXiv:2607.20064, July 2026. [11] Explore Before You Solve: The Speed--Depth Trade-off in Epistemic Agents for ARC-AGI-3. arXiv:2605.25931, May 2026. [12] Kaggle discussion thread on milestone-2 approaches, topic 744792. kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/744792 [13] Jolicoeur-Martineau, A. Less is More: Recursive Reasoning with Tiny Networks. ARC Prize 2025 paper award, 1st. arXiv:2510.04871. [14] Pourcel, J., Colas, C., Oudeyer, P.-Y. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI. ARC Prize 2025 paper award, 2nd. ICML 2025; arXiv:2507.14172; OpenReview z4IG090qt2. [15] Liao, I., Gu, A. ARC-AGI Without Pretraining. ARC Prize 2025 paper award, 3rd. [16] Li, W.-D., Ellis, K., et al. Combining Induction and Transduction for Abstract Reasoning. ARC Prize 2024 paper award, 1st. arXiv:2411.02272. [17] Akyürek, E., Damani, M., Zweiger, A., et al. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. ARC Prize 2024 paper award, 2nd. arXiv:2411.07279. [18] Bonnet, C., Macfarlane, M. Searching Latent Program Spaces. ARC Prize 2024 paper award, 3rd. [19] Ariño de la Rubia, E., Pafka, S. Identical Runs, Different Results: Benchmarking AI Coding Agents on Open-Weight Models. arXiv:2609.33812, September 2026. [20] Masters, C., Liu, Z., Albrecht, S. V. Rollout Cards: A Reproducibility Standard for Agent Research. arXiv:2605.12131, May 2026.

Appendix A. The screen protocol, and the traps

Artifact layout. One run directory per batch contains a `serve.log`, one worker directory per arm with `metrics.json`, `score.json`, `request-events.jsonl` and the transcripts, and a `batch-status.json` recording the state of every arm. A batch that trips its safety bound stops with partial artifacts; those arms are labeled

incomplete and excluded from means rather than patched with imputed values. Every table in this paper is regenerated by `tools/build_numbers.py`, which cites the artifact path per number.

Budget arithmetic. A six-arm screen costs `setup + 6 x (game budget + worker overhead) + teardown`. With the 5400-second setup ceiling, 2000 seconds per game and a 260-second overhead, the ceiling arithmetic is 5400 + 6 x 2260 = 18,960 seconds, or 5 hours 16 minutes, before teardown; with the typical 25-minute model load the same screen lands near 3 hours 50 minutes. The alternating repeat matrix (E7) uses the same arithmetic: six arms, three of them identical baselines, alternating A-B-A-B-A-B. Each candidate sits directly after the arm it is compared with, which controls position; the design does not close the bracket on a final baseline, so we read it as three paired deltas plus three same-configuration samples rather than as a fully bracketed screen. The batch stops early when its own deadline nears, which is why aborted screens have partial arms rather than a corrupted total.

Six traps we hit, and the fix.

1. A model load under storage contention can take 75 minutes instead of 10. Fix: a 5400-second setup ceiling and a readiness probe that checks the server answers one request before any arm starts.
2. A shared-cache flush probe with a 15-second budget aborts the whole batch when the cache host is slow. Fix: a bounded retry on the probe, and a batch status file that records the failure reason instead of killing the run.
3. Notebook cells serialized without their trailing newlines turn a whole cell into a single comment line. Fix: a serializer that preserves line endings, plus a `compile()` check on every generated notebook before it is pushed.
4. Arm definitions that are not all the same tuple shape fail at `ARMS[arm][2]` deep inside the batch. Fix: a self-check that asserts every arm is a `(passes, budget, overrides)` triple before any push.
5. Status watchers that match substrings can read a network error text as a terminal state. Fix: match exact status tokens only, and treat everything else as still running.
6. Kaggle file downloads fail from this host over one path and succeed over another: the list API answers only on a direct connection, the file host only through the local proxy. Fix: two-phase fetching with opposite network settings, and a retry ladder on both phases.

Decision rules. A candidate wins only if its margin over the end control exceeds the session's observed variation; the rule was applied retrospectively to the screens that predate the protocol and used prospectively after it. After the screen, the label is chosen from the four in Table 3 and cannot be upgraded without new samples. Estimates are recomputed from artifacts before any claim leaves the analysis, and negative rows stay in the tables.

Appendix B. Per-game tables

Generated from the run artifacts by `tools/build_appendix_b.py`. Cells read `score L levels A actions E errors`. Means are calculated from unrounded scores, so a mean can differ by 0.01 to 0.02 from the average of the displayed one-decimal cells. The full JSON for every arm is in the release.

Screen E5 (`phase_b_safe_1791857607678366179`)

Game	baseline	noop_guard
ar25	27.8 L4 A134 E5	41.7 L5 A140 E3
bp35	0.0 L0 A34 E4	0.8 L1 A51 E6
cd82	4.8 L1 A46 E11	4.8 L1 A20 E9
cn04	4.8 L1 A42 E3	4.8 L1 A34 E10
dc22	0.0 L0 A63 E5	14.3 L2 A101 E5
ft09	47.6 L4 A58 E5	100.0 L6 A92 E0
g50t	0.0 L0 A13 E13	0.0 L0 A44 E8
ka59	3.6 L1 A41 E8	10.7 L2 A64 E9
lf52	1.8 L1 A40 E9	1.8 L1 A45 E5
lp85	38.8 L5 A81 E5	41.7 L5 A67 E7
Mean	**12.91**	**22.04**

Screen E5b (`phase_b_safe_1791871401644733360`)

Game	baseline	baseline_rep	drain60	inventory	memory_slim	noop_guard
ar25	27.8 L4 A102 E8	27.8 L4 A109 E10	41.7 L5 A120 E6	27.8 L4 A115 E6	25.8 L4 A127 E6	8.3 L2 A61 E12
bp35	2.2 L1 A17 E6	1.0 L1 A62 E5	2.2 L1 A28 E8	0.0 L0 A10 E12	1.9 L1 A31 E8	2.2 L1 A26 E6
cd82	0.0 L0 A41 E8	0.0 L0 A23 E12	14.3 L2 A53 E10	14.3 L2 A79 E4	14.3 L2 A69 E3	44.7 L4 A110 E2
cn04	4.8 L1 A22 E10	4.8 L1 A39 E10	3.9 L1 A42 E6	14.3 L2 A67 E4	4.8 L1 A43 E12	14.3 L2 A61 E4
dc22	14.3 L2 A121 E5	1.7 L1 A104 E8	0.0 L0 A19 E15	14.3 L2 A100 E5	14.3 L2 A109 E4	3.9 L1 A93 E3
ft09	47.6 L4 A95 E5	100.0 L6 A88 E2	100.0 L6 A81 E0	28.6 L3 A57 E12	47.6 L4 A76 E1	100.0 L6 A92 E3
g50t	3.6 L1 A64 E5	0.0 L0 A60 E3	0.0 L0 A41 E5	3.6 L1 A51 E5	0.0 L0 A54 E6	0.0 L0 A18 E11
ka59	3.6 L1 A21 E13	3.6 L1 A22 E12	3.6 L1 A31 E8	3.6 L1 A46 E12	3.6 L1 A21 E14	3.6 L1 A19 E12
lf52	5.5 L2 A67 E7	1.8 L1 A42 E5	1.8 L1 A65 E2	1.8 L1 A65 E4	1.8 L1 A36 E7	5.5 L2 A95 E4
lp85	39.7 L5 A85 E4	41.7 L5 A106 E1	41.7 L5 A66 E3	27.8 L4 A59 E9	41.7 L5 A69 E4	21.8 L4 A77 E5
Mean	**14.90**	**18.22**	**20.91**	**13.59**	**15.56**	**20.43**

Screen E6 (`phase_b_safe_1791952567840217146`)

Game	baseline	baseline_rep	combo	drain60	noop_guard
ar25	27.8 L4 A108 E8	16.7 L3 A100 E5	27.8 L4 A121 E5	27.8 L4 A116 E4	27.8 L4 A108 E8
bp35	2.2 L1 A23 E6	0.0 L0 A17 E11	0.0 L0 A15 E11	2.0 L1 A29 E5	0.0 L0 A40 E8
cd82	0.0 L0 A31 E10	4.8 L1 A37 E14	38.4 L4 A98 E3	14.3 L2 A45 E8	24.6 L3 A87 E6
cn04	4.8 L1 A31 E9	4.8 L1 A25 E5	14.3 L2 A68 E5	14.3 L2 A50 E3	4.8 L1 A56 E4
dc22	0.0 L0 A123 E5	4.8 L1 A48 E5	0.0 L0 A131 E6	14.3 L2 A106 E6	0.0 L0 A23 E13
ft09	47.6 L4 A86 E4	47.6 L4 A67 E7	47.6 L4 A76 E4	47.6 L4 A65 E8	100.0 L6 A85 E2
g50t	3.6 L1 A45 E9	0.0 L0 A31 E10	0.0 L0 A30 E11	0.0 L0 A30 E7	0.0 L0 A17 E11
ka59	10.7 L2 A101 E5	21.4 L3 A164 E6	2.2 L1 A38 E7	3.3 L1 A33 E13	3.6 L1 A75 E7
lf52	1.8 L1 A40 E6	5.5 L2 A72 E5	1.8 L1 A57 E5	1.8 L1 A34 E7	1.8 L1 A58 E7
lp85	16.7 L3 A43 E5	41.7 L5 A64 E4	24.1 L4 A74 E3	41.7 L5 A61 E5	41.7 L5 A101 E4
Mean	**11.52**	**14.71**	**15.61**	**16.71**	**20.42**

Screen E7 (`phase_b_safe_1791969354283162683`)

Game	base_1	base_2	base_3	guard_1	guard_2	guard_3
ar25	27.8 L4 A123 E4	27.8 L4 A124 E7	26.7 L4 A121 E7	27.8 L4 A106 E6	27.8 L4 A104 E7	27.8 L4 A116 E8
bp35	2.2 L1 A30 E2	1.4 L1 A35 E5	0.0 L0 A16 E6	2.2 L1 A26 E5	2.2 L1 A24 E7	0.0 L0 A15 E8
cd82	6.3 L2 A62 E5	2.9 L1 A70 E5	0.0 L0 A38 E12	11.6 L2 A51 E9	47.6 L4 A90 E4	0.0 L0 A35 E10
cn04	0.0 L0 A30 E7	14.3 L2 A58 E9	4.8 L1 A86 E6	14.3 L2 A51 E5	14.3 L2 A68 E3	0.0 L0 A23 E11
dc22	4.5 L1 A61 E10	4.8 L1 A57 E4	0.0 L0 A73 E11	4.8 L1 A53 E6	0.0 L0 A48 E9	14.3 L2 A152 E5
ft09	28.6 L3 A49 E13	47.6 L4 A56 E3	47.6 L4 A75 E9	45.3 L4 A72 E5	47.6 L4 A55 E9	47.6 L4 A72 E7
g50t	0.0 L0 A22 E9	0.0 L0 A67 E8	0.0 L0 A45 E9	0.0 L0 A19 E11	0.0 L0 A78 E8	3.6 L1 A48 E6
ka59	10.7 L2 A93 E4	3.6 L1 A29 E14	10.7 L2 A76 E8	3.6 L1 A25 E13	3.6 L1 A40 E10	3.6 L1 A28 E12
lf52	5.5 L2 A78 E7	5.1 L2 A101 E5	5.5 L2 A81 E3	5.5 L2 A87 E3	1.8 L1 A64 E5	1.8 L1 A94 E2
lp85	41.7 L5 A69 E6	27.8 L4 A59 E5	27.8 L4 A53 E7	39.4 L5 A107 E1	41.7 L5 A68 E4	25.6 L4 A69 E2
Mean	**12.71**	**13.53**	**12.30**	**15.43**	**18.66**	**12.43**

ft09 per arm across all four screens

Screen	Arm	Group	ft09 score	Levels	Outcome
E5	noop_guard	guard	100.00	6	solve
E5	baseline	non-guard	47.62	4	
E5b	noop_guard	guard	100.00	6	solve
E5b	baseline	non-guard	47.62	4	
E5b	baseline_rep	non-guard	100.00	6	solve
E5b	drain60	non-guard	100.00	6	solve
E5b	inventory	non-guard	28.57	3	
E5b	memory_slim	non-guard	47.62	4	
E6	noop_guard	guard	100.00	6	solve
E6	baseline	non-guard	47.62	4	
E6	baseline_rep	non-guard	47.62	4	
E6	combo	non-guard	47.62	4	
E6	drain60	non-guard	47.62	4	
E7	guard_1	guard	45.27	4	
E7	guard_2	guard	47.62	4	
E7	guard_3	guard	47.62	4	
E7	base_1	non-guard	28.57	3	
E7	base_2	non-guard	47.62	4	
E7	base_3	non-guard	47.62	4	

Guard arms: 6, ft09 solves: 3. Non-guard arms: 13, ft09 solves: 2. Inclusion rule for the second denominator: every arm of the four screens with a per-game record (E5 has two completed arms; E5b six; E6 five; E7 six).

Calibration runs (25 games each)

Game	a1	a2	a3	a4	a5
ar25	0.0 L0 A0 E0	77.8 L7 A228 E0	41.7 L5 A145 E6	100.0 L8 A272 E9	41.7 L5 A142 E1
bp35	0.0 L0 A0 E0	6.4 L2 A86 E4	0.8 L1 A43 E10	2.2 L1 A23 E9	0.8 L1 A54 E1
cd82	0.0 L0 A0 E0	28.6 L3 A101 E3	47.6 L4 A110 E8	0.0 L0 A63 E0	14.3 L2 A77 E1
cn04	0.0 L0 A0 E0	14.3 L2 A85 E4	4.8 L1 A40 E12	4.8 L1 A31 E12	28.6 L3 A103 E2
dc22	0.0 L0 A0 E0	12.6 L2 A173 E1	4.6 L1 A76 E12	14.3 L2 A139 E3	14.3 L2 A140 E1
ft09	0.0 L0 A0 E0	100.0 L6 A91 E0	47.6 L4 A51 E15	100.0 L6 A98 E6	100.0 L6 A94 E0
g50t	0.0 L0 A0 E0	10.7 L2 A98 E4	3.6 L1 A62 E8	0.0 L0 A40 E5	0.0 L0 A57 E6
ka59	0.0 L0 A0 E0	3.6 L1 A39 E12	3.6 L1 A35 E18	10.7 L2 A122 E7	1.0 L1 A62 E7
lf52	0.0 L0 A0 E0	3.5 L2 A142 E1	5.5 L2 A83 E10	1.8 L1 A58 E4	1.8 L1 A60 E1
lp85	0.0 L0 A0 E0	100.0 L8 A147 E0	41.7 L5 A95 E8	77.8 L7 A123 E26	36.0 L5 A155 E1
ls20	0.0 L0 A0 E0	10.7 L2 A144 E0	1.6 L1 A50 E8	3.6 L1 A87 E6	2.1 L1 A86 E2
mor0	0.0 L0 A0 E0	0.0 L0 A0 E0	13.0 L2 A115 E4	28.6 L3 A131 E7	14.3 L2 A100 E2
r11l	0.0 L0 A0 E0	0.0 L0 A0 E0	4.8 L1 A19 E6	4.8 L1 A21 E6	4.8 L1 A26 E1
re86	0.0 L0 A0 E0	0.0 L0 A0 E0	41.7 L5 A234 E5	41.7 L5 A300 E4	41.7 L5 A264 E0
s5i5	0.0 L0 A0 E0	0.0 L0 A0 E0	2.8 L1 A32 E20	8.3 L2 A74 E11	2.5 L1 A37 E9
sb26	0.0 L0 A0 E0	0.0 L0 A0 E0	77.8 L7 A136 E13	100.0 L8 A136 E0	100.0 L8 A143 E0
sc25	0.0 L0 A0 E0	0.0 L0 A0 E0	23.9 L3 A202 E4	44.2 L4 A198 E6	47.6 L4 A169 E2
sk48	0.0 L0 A0 E0	0.0 L0 A0 E0	0.0 L0 A23 E19	2.8 L1 A30 E12	0.0 L0 A53 E5
sp80	0.0 L0 A0 E0	0.0 L0 A0 E0	28.6 L3 A59 E7	14.3 L2 A48 E31	4.8 L1 A34 E6
su15	0.0 L0 A0 E0	0.0 L0 A0 E0	13.3 L3 A78 E11	6.7 L2 A50 E5	13.3 L3 A77 E1
tn36	0.0 L0 A0 E0	0.0 L0 A0 E0	3.6 L1 A113 E3	3.6 L1 A34 E8	10.7 L2 A180 E2
tr87	0.0 L0 A0 E0	0.0 L0 A0 E0	100.0 L6 A375 E0	3.2 L1 A195 E6	70.5 L5 A423 E1
tu93	0.0 L0 A0 E0	0.0 L0 A0 E0	62.2 L7 A245 E2	11.8 L3 A63 E27	26.1 L5 A226 E0
vc33	0.0 L0 A0 E0	0.0 L0 A0 E0	100.0 L7 A240 E0	35.7 L4 A110 E19	21.4 L3 A103 E1
wa30	0.0 L0 A0 E0	0.0 L0 A0 E0	10.5 L3 A584 E1	13.3 L3 A208 E6	6.7 L2 A229 E0
Mean	**0.00 (25 games)**	**14.72 (25 games)**	**27.40 (25 games)**	**25.36 (25 games)**	**24.20 (25 games)**

Appendix C. Experiment registry

Every experiment started over the competition, its state, and why it did or did not enter the analysis. Kernel slugs point at the pushed notebooks in the release.

id	kernel	purpose	design	state	disposition
E4	-	Serving-stack throughput probe	archived notebook and module only	code archived, no retained run artifacts	informed the calibration-run design; not an analysis input
a1	`zgyyyl/arc3-calib-a`	First calibration probe, 25 games, single arm	single execution	all 25 game records cancelled with zero actions	excluded: no usable play records
a2	`zgyyyl/arc3-calib-a2`	Calibration follow-up, 25 games, single arm	single execution	complete (2026-10-12), mean 14.72	latency context; Appendix B
a3	`zgyyyl/arc3-calib-a3`	Calibration run, 25 games, stock 120-second analyzer timeout	single execution	complete (2026-10-12), mean 27.40	member of the timeout pair (a3 vs a5), Table 2
a4	`zgyyyl/arc3-calib-a4`	Calibration run, 25 games, widened per-game ceiling and session count	single execution	complete (2026-10-12), mean 25.36	latency anatomy pair (a4 vs a5); queue-delay statistics
a5	`zgyyyl/arc3-calib-a5`	Calibration run, 25 games, doubled analyzer timeout 240 seconds	single execution	complete (2026-10-12), mean 24.20	member of the timeout pair and the latency pair
E5	`zgyyyl/arc3-e5-behavior`	Behavior screen: baseline vs no-op guard	bracketed screen, batch aborted	two of the planned arms completed; batch stopped by a watcher failure	both completed arms enter Table 1, Table 4 and the ft09 count; the lost arms were re-run as E5b
E5b	`zgyyyl/arc3-e5b-behavior`	Behavior screen re-run: six arms including inventory, memory slim, drain and a repeat baseline	fully bracketed screen, six arms	complete (2026-10-13)	main screen for the guard, inventory, memory-slim and drain readings, Tables 1-4
E6	`zgyyyl/arc3-e6-combo`	Second behavior screen: baseline, guard, drain, combo, repeat baseline	fully bracketed screen, five arms	complete (2026-10-14); the first kernel build failed before running and was rebuilt	second controlled guard reading and the drain/combo candidates, Tables 1-4
E6-b1	`zgyyyl/arc3-e6-combo`	E6 kernel, first build: failed before any run (serialization error in one arm's configuration)	build only	failed at build stage; no records produced	rebuilt as E6; the failed build contributes nothing to the analysis
E7-b1	`zgyyyl/arc3-e7-repeat-matrix`	E7 kernel, first build: failed budget validation before any run	build only	failed at build stage; no records produced	rebuilt as E7; the failed build contributes nothing to the analysis
E7	`zgyyyl/arc3-e7-repeat-matrix`	Alternating repeat matrix: three baseline and three guard arms interleaved	A-B-A-B-A-B, six arms, ten games each	complete (2026-10-15); the first kernel build failed budget validation and was rebuilt	same-configuration spread for both arms of a pair, Section 4 and the guard paragraphs
S-noop	`zgyyyl/arc-agi-3-daniel-m2-baseline`	Submission build: baseline plus no-op guard only	local build of the competition kernel	built and locally verified	not submitted; superseded by the three-switch guard build
S-guard	`zgyyyl/arc-agi-3-daniel-m2-baseline`	Submission build: three guard switches on	local build of the competition kernel	submitted twice: 57147727 (2026-10-13) and 57174905 (2026-10-14)	the linked submission of this paper, Table 3a

id	kernel	purpose	design	state	disposition
S-drain	`zgyyyl/arc-agi-3-daniel-m2-baseline`	Submission build: guard build plus history turn drain at 60 turns	local build of the competition kernel	built and locally verified; local deltas +5.19 (E6) and +2.00 (end-control view)	not submitted: daily submission limit and an unresolved reading
S-combo	`zgyyyl/arc-agi-3-daniel-m2-baseline`	Submission build: guards plus drain combined	local build of the competition kernel	built and locally verified	not submitted: weak local reading
N1	-	Overnight orchestration script (downloads, pushes, watchers)	infrastructure, not an experiment	log retained	not an analysis input

The two failed builds above never produced records: the failure happened before the batch started, so neither screen contains partial arms from them.